

MIĘDZYNARODOWY SYSTEM KODÓW POCZTOWYCH
INTERNATIONAL POSTCODE SYSTEM (IPCS)

OPIS I DOKUMENTACJA TECHNICZNA PROJEKTU

AUTOR

BARTOSZ CIEŚLICKI

Założenia systemu

Główną koncepcją systemu jest stworzenie jednolitego systemu kodów pocztowych dla całej ziemi. Najważniejszy cel to:

ZAKODOWANIE KOORDYNATÓW GEOGRAFICZNYCH KAŻDEGO KODU POCZTOWEGO W SAMYM JEGO ZAPISIE.

System tak skonstruowany, wyeliminuje konieczność posiadania dużej ogólnoswiatowej bazy danych zawierającej lokalizację każdego kodu pocztowego, zmniejszy bądź całkowicie zlikwiduje koszty geokodowania oraz ogromnie przyspieszy systemy komputerowe które wyszukują pozycję geograficzną dla żadanego adresu.

Założenia:

- 1) Powierzchnia, którą identyfikuje kod pocztowy nie powinna przekraczać 1000m². Tak, aby można było w łatwy i precyzyjny sposób dotrzeć do pożądanego miejsca, używając mapy bądź nawigacji satelitarnej.
- 2) System musi obejmować każdą część planety, bez względu na zaludnienie, gęstość zabudowy, ukształtowanie czy rodzaj powierzchni.
- 3) Kody pocztowe muszą być wyliczane/tworzone na podstawie algorytmu, a nie nadawane.
- 4) Z każdego kodu pocztowego można wyliczyć koordynaty geograficzne wskazujące środek powierzchni którą identyfikuje kod pocztowy.

Zapis Międzynarodowego Kodu Pocztowego (IPCS) :

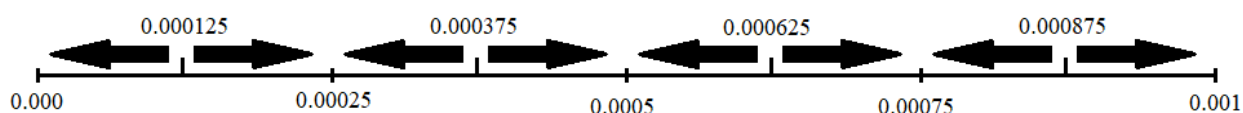
- 1) Powinna opierać się na alfabecie łacińskim, jako najpopularniejszym na świecie.
- 2) Powinna zawierać relatywnie małą liczbę znaków, pozwalającą na łatwe zapamiętanie.
- 3) Nie powinna zawierać znaków specjalnych charakterystycznych dla danego kraju.
- 4) Powinna eliminować najczęstsze pomyłki w zapisie i odczycie (czyli litery O z liczbą 0 oraz litery I z liczbą 1).

Konstrukcja sektorów

Do wyliczenia Międzynarodowego Kodu Pocztowego (IPCS), wykorzystamy długość i szerokość geograficzną zapisaną w systemie dziesiętnym. Długość geograficzna jest z przedziału $(-180, 180>$ natomiast szerokość geograficzna z przedziału $(-90, 90 >$. Pierwszym bardzo istotnym krokiem do obliczenia Międzynarodowego Kodu Pocztowego jest zaokrąglenie długości i szerokości geograficznej do ćwiartki liczby 0.001.

Przykład:

Założmy, że liczba $t \in <0.000, 0.001 >$, liczba x to zaokrąglenie.

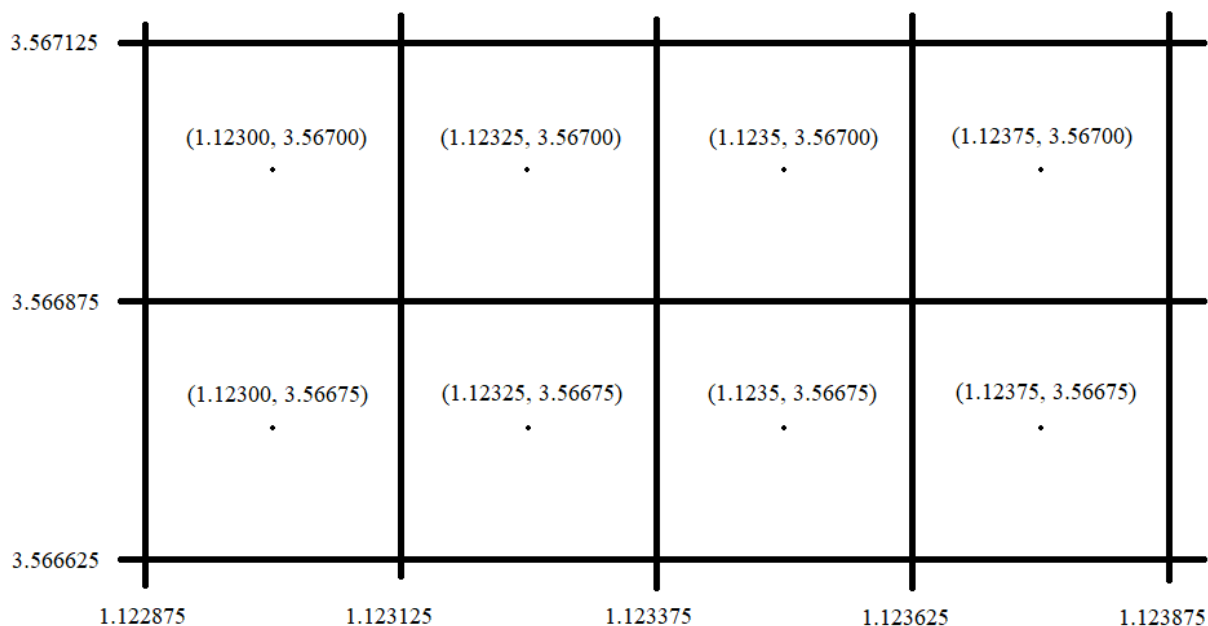


Jeżeli

- $t \in <0.000, 0.000125 >$ to $x = 0.000$
- $t \in <0.000125, 0.000375 >$ to $x = 0.00025$
- $t \in <0.000375, 0.000625 >$ to $x = 0.0005$
- $t \in <0.000625, 0.000875 >$ to $x = 0.00075$
- $t \in <0.000875, 0.001 >$ to $x = 0.001$

Dzięki zastosowaniu zaokrągleń cała ziemia zostaje podzielona na strefy, a otrzymany jeden koordynat, identyfikuje nam każdy powstały sektor na ziemi i jest jednocześnie jego środkiem. Spełnia to założenie, które postawiliśmy na początku.

Wycinek siatki wygląda następująco:



Budowa sektorów opartych na koordynatach powoduje, że powierzchnia każdego obszaru powstałego z zaokrąglenia długości i szerokości geograficznej, będzie malała wraz z przesuwaniem się do biegunów ziemi. Największe powierzchnie będą zatem na równiku i będą wynosiły około 728m², co spełnia założenie.

Możemy obliczyć ilość powstałych w ten sposób sektorów. Liczby, są zaokrąglane do ćwiartki tysięcznej, zatem każda tysięczna jest podzielona na 4. Dla długości geograficznej mamy przedział liczbowy (-180, 180>. Przez x oznaczmy ilość podziałów. Wynosi ona:

$$x = 360 * 1000 * 4 \quad (1)$$

Dla szerokości geograficznej mamy przedział liczbowy <-90, 90 >. Przez y oznaczmy ilość podziałów. Wynosi ona:

$$y = 181 * 1000 * 4 \quad (2)$$

(Wartość 181, zamiast 180, uwzględnia fakt, że przedział szerokości geograficznej <-90, 90> jest domknięty obustronnie — biegun południowy (-90°) i biegun północny (90°) to dwa różne punkty — w przeciwieństwie do przedziału długości geograficznej (-180, 180>, który jest domknięty jednostronnie, ponieważ -180° i 180° oznaczają ten sam południk.)

Korzystając z (1) i (2) obliczamy ilość sektorów dla całej ziemi :

$$\text{Ilość sektorów} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

Pozostaje nam jedynie konstrukcja Międzynarodowego Kodu Poczтового (IPCS) dla każdego powstałego sektora.

Konstrukcja Międzynarodowego Kodu Pocztowego (IPCS) :

Używając cyfr i alfabetu łacińskiego mamy do dyspozycji 10 cyfr oraz 26 liter. Z alfabetu usuwamy litery O oraz I (aby spełnić założenia systemu). Czyli dysponujemy 34 następującymi znakami { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z }. Bazując na 34 znakach tworzymy system liczbowy, trzydziesto-czwórkowy.

Po lewej stronie mamy cyfry w systemie trzydziesto-czwórkowym, po prawej odpowiednik w systemie dziesiętnym.

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

(4)

Każdą liczbę bazującą na 34 znakach, możemy przedstawić w systemie dziesiętnym w następujący sposób:

$\dots dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + \dots$, gdzie f jest funkcją przypisującą każdej z cyfr z systemu trzydziesto-czwórkowego, jego odpowiednika w systemie dziesiętnym, (patrz (4)). Natomiast a, b, c, d są cyframi z systemu trzydziesto-czwórkowego.

Przykład:

$$\begin{aligned} AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 = \\ &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 = \\ &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 = \\ &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007 \end{aligned}$$

Jak możemy zauważyć zapis trzydziesto-czwórkowy jest znacznie krótszy. Chcąc wygenerować kody pocztowe dla całej planety będziemy potrzebowali 8 znaków. Maksymalna ilość znaków w obowiązującym, angielskim i irlandzkim systemie kodów pocztowych to 7, w japońskim 7 cyfr, holenderskim 6 znaków, itd. Biorąc pod uwagę długość kodów pocztowych w innych krajach i fakt, że Międzynarodowy System Kodów Pocztowych jest systemem globalnym, przyjmujemy, że 8 znaków jest akceptowalne i spełnia założenia systemu.

W naszym trzydziesto-czwórkowym systemie największą liczbą którą możemy zapisać, używając 8 cyfr z tegoż systemu to (szczegółowe wyliczenia zostały pominięte):

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

Wynika z tego, że Ziemia może posiadać 1 785 793 904 896 kodów pocztowych (ilość została powiększona o 1 względem największej liczby ponieważ 00000000 również może być kodem). Z wcześniejszego wyliczenia (3), znamy ilość sektorów w konstrukcji i jest ona równa liczbie 1 042 560 000 000. Liczba ta jest mniejsza od 1 785 793 904 895 zatem, 8 cyfr z systemu trzydziesto-czwórkowego pozwoli na jednoznacznie określenie każdego sektora. Sektory nie są równej wielkości, ze względu na samą konstrukcję. Największe, znajdują się w okolicach równika i wynoszą $\approx 728\text{m}^2$. Ziemia ma powierzchnię około $510\,100\,000\text{ km}^2 = 510\,100\,000\,000\,000\text{ m}^2$. Chcąc obliczyć średnią powierzchnię, zajmowaną przez sektor, wykonujemy proste działanie:

$$510\,100\,000\,000\,000\text{ m}^2 / 1\,042\,560\,000\,000 \approx 489.2764\text{ m}^2$$

Czyli liczba $ZZZZZZZZ_{34}$ spełnia założenia, a powierzchnia każdego sektora jest mniejsza od $1000m^2$.

Pozostaje nam konwersja środka sektorów na Międzynarodowy Kod Pocztowy (IPCS).

Dla naszych potrzeb część całkowitą liczb ujemnych będziemy przedstawiali w następujący sposób:

Dla długości geograficznej, ujemną część całkowitą x zamienimy na $360 + x$.

Dla szerokości geograficznej, ujemną część całkowitą y zamienimy na $180 + y$.

Część ułamkowa pozostaje bez zmian.

Podsumowując:

Długość geograficzna $-XXX.xxxxx$ zostanie zamieniona na $(360 + (-XXX)).xxxxx$

Szerokość geograficzna $-YYY.yyyyy$ zostanie zamieniona na $(180 + (-YYY)).yyyyy$

Przykłady zamiany:

Liczba -075.43725 dla długości geograficznej jest równa 285.43725 , bo $360 + (-75) = 285$

Liczba -000.411250 dla długości geograficznej jest równa 360.411250 , bo $360 + 0 = 360$

Liczba -015.44375 dla szerokości geograficznej jest równa 165.44375 , bo $180 + (-15) = 165$

Przy tworzeniu algorytmu należy wziąć pod uwagę fakt, że w tym zapisie, biegun południowy 90 stopni szerokości geograficznej i biegun północny -90 stopni szerokości geograficznej, są sobie równe.

Uwzględniając zapis liczb ujemnych oraz zaokrąglenia, ogólna postać koordynatów zapisanych w systemie dziesiętnym ma postać:

Dla długości geograficznej : $XXX.xxxqq$, gdzie $XXX \in \{x \in \mathbb{N} : 0 \leq x \leq 360\}$,
 $xxx \in \{x \in \mathbb{N} : 0 \leq x \leq 999\}$,
 $qq \in \{00, 25, 50, 75\}$

Dla szerokości geograficznej : $YYY.yyygg$,gdzie $YYY \in \{x \in \mathbb{N} : 0 \leq x \leq 180\}$,
 $yyy \in \{x \in \mathbb{N} : 0 \leq x \leq 999\}$,
 $gg \in \{00, 25, 50, 75\}$

W kolejnym kroku, każdy człon liczby $XXX.xxxqq$ oraz liczby $YYY.yyygg$ konwertujemy na system dwójkowy.

$XXX_{10} = XXXXXXXXX_2$ - największa liczba 360. Do zapisu tej liczby potrzebujemy 9 bitów.

$xxx_{10} = xxxxxxxxx_2$ - największa liczba 999. Do zapisu tej liczby potrzebujemy 10 bitów.

$qq_{10} = qq_2$ - największa liczba 75. Jednakże, nasz zbiór ma 4 elementy dlatego konwersja polega na przypisaniu $00_{10} \Rightarrow 00_2$, $25_{10} \Rightarrow 01_2$, $50_{10} \Rightarrow 10_2$, $75_{10} \Rightarrow 11_2$

$YYY_{10} = YYYYYYYY_2$ - największa liczba 180. Do zapisu tej liczby potrzebujemy 8 bitów.

$yyy_{10} = yyyyyyyyy_2$ - największa liczba 999. Do zapisu tej liczby potrzebujemy 10 bitów.

$gg_{10} = gg_2$ - największa liczba 75. Jednakże, nasz zbiór ma 4 elementy dlatego konwersja polega na przypisaniu $00_{10} \Rightarrow 00_2$, $25_{10} \Rightarrow 01_2$, $50_{10} \Rightarrow 10_2$, $75_{10} \Rightarrow 11_2$

Następnie ustawiamy liczby z systemu dwójkowego w następującej kolejności:

$$\text{XXXXXXXXX}_2 \text{ YYYYYYYY}_2 \text{ xxxxxxxxxxx}_2 \text{ yyyyyyyyyy}_2 \text{ qq}_2 \text{ gg}_2$$

Bierzemy liczbę minimum 41 bitową, zerujemy ją i wpisujemy do niej powyższe bity od prawej strony do lewej zachowując ich kolejność. Otrzymana liczba jest równa:

$$\text{XXXXXXXXXXYYYYYYYxxxxxxxxxyyyyyyyyyqqgg}_2$$

W ostatnim kroku zamieniamy system dwójkowy na trzydziesto-czwórkowy. Otrzymana liczba trzydziesto-czwórkowa jest jednocześnie naszym Międzynarodowym Kodem Pocztowym (IPCS). Kryjąc w swoim zapisie koordynaty geograficzne, środka każdego sektora.

Kolejność zapisu bitów jest bardzo istotna, ponieważ zaokrąglony i zapisywany zgodnie z konwencją, koordynat o największej wartości liczbowej wynosi 360.99975, 180.99975. Konwertując go na system dwójkowy otrzymujemy liczbę:

$$1011010001011010011111001111111001111111_2 = 1\,549\,224\,509\,055_{10}$$

Otrzymana liczba jest mniejsza od maksymalnej liczby z kodu trzydziesto-czwórkowego który wynosi $\text{ZZZZZZZ}_{34} = 1\,785\,793\,904\,895_{10}$. Zamieniając kolejność bitów możemy przekroczyć wartość maksymalną, co uniemożliwi stworzenie kodów pocztowych dla wszystkich sektorów.

Aby, otrzymać koordynaty środka sektora / Międzynarodowego Kodu Pocztowego wykonujemy działania w odwrotnej kolejności.

Przykłady:

Międzynarodowy Kod Pocztowy (IPCS), dla koordynatów -1.347120, 53.983489 konstruujemy następująco:

- 1) Zaokrąglamy, konwertujemy koordynaty i otrzymujemy 359.34700, 53.98350
- 2) Zamieniamy każdy człon koordynatu, czyli liczby 359, 347, 00, 53, 983, 50 na system dwójkowy zgodnie z powyższym opisem. I otrzymujemy 101100111_2 , 0101011011_2 , 00_2 , 00110101_2 , 1111010111_2 , 10_2
- 3) Ustawiamy liczby w odpowiedniej kolejności 101100111_2 , 00110101_2 , 0101011011_2 , 1111010111_2 , 00_2 , 10_2
- 4) Tworzymy liczbę $101100111001101010101010110111110101110010_2$
- 5) Konwertujemy liczbę dwójkową na trzydziesto-czwórkową i otrzymujemy VCPM6TKY_{34}
- 6) Otrzymana liczba jest Międzynarodowym Kodem Pocztowym (IPCS): **VCPM 6TKY**

Koordynaty dla Międzynarodowego Kodu Pocztowego 1GL8 YT7F obliczamy następująco:

- 1) Konwertujemy liczbę trzydziesto-czwórkową $1\text{GL}8\text{YT}7\text{F}_{34}$ na dwójkową i otrzymujemy $00001001000110010110000001101001101101001_2$
- 2) Wyodrębniamy liczby 000010010_2 , 00110010_2 , 1100000011_2 , 0100110110_2 , 10_2 , 01_2
- 3) Zamieniamy kolejność 000010010_2 , 1100000011_2 , 10_2 , 00110010_2 , 0100110110_2 , 01_2
- 4) Zamieniamy liczby dwójkowe na dziesiętne 18, 771, 50, 50, 310, 25
- 5) Z otrzymanych liczb otrzymujemy koordynat **18.77150, 50.31025**

Funkcja generująca i obliczająca koordynaty z Międzynarodowego Kodu Pocztowego:

Poniżej znajdują się 2 funkcje napisane w języku C#. Pierwsza generuje Międzynarodowy Kod Pocztowy, a druga oblicza koordynaty z Międzynarodowego Kodu Pocztowego.

```

public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Codes = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
"A", "B", "C", "D", "E", "F", "G", "H", "J", "K", "L", "M", "N", "P", "Q", "R", "S",
"T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
                TempHiLat = TempHiLat | OptionsLat;
            }

            TempLoLat = TempLoLat << 4;
        }
    }
}

```

```

        ulong OptionsLon = 5;
        Precision = Convert.ToInt32(StrsLon[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLon = 0;
        }
        else if (Precision < 375)
        {
            OptionsLon = 1;
        }
        else if (Precision < 625)
        {
            OptionsLon = 2;
        }
        else if (Precision < 875)
        {
            OptionsLon = 3;
        }

        ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
        TempHiLon = TempHiLon << 32;
        ulong TempLoLo = 0;
        if (StrsLon.Length > 1)
        {
            TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
            if (OptionsLon == 5)
            {
                TempLoLo++;
            }
            else
            {
                TempHiLon = TempHiLon | (OptionsLon * 4);
            }

            TempLoLo = TempLoLo << 14;
        }

        ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

        ulong Pos = 0;
        string Res = "";
        for (int i = 7; i >= 0; i--)
        {
            Pos = UlongResHi / (ulong)Math.Pow(34, i);
            UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
            Res += Coder[Pos];
        }

        return Res;
    }
    catch
    {
        return "";
    }
}

```

```

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {
        Postcode = Postcode.ToUpper();
        string Coder = "0123456789ABCDEFGHIJKLMNPQRSTUVWXYZ";
        ulong Hi = 0;

        for (int i = 0; i < 8; i++)

```

```

- i));
    {
        Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7
    }

    ulong OptionLat = (Hi & 3);
    double HalfLat = 0;
    switch (OptionLat)
    {
        case 1:
            HalfLat = 0.00025;
            break;
        case 2:
            HalfLat = 0.0005;
            break;
        case 3:
            HalfLat = 0.00075;
            break;
    }

    ulong OptionLon = (Hi & 12) >> 2;
    double HalfLon = 0;
    switch (OptionLon)
    {
        case 1:
            HalfLon = 0.00025;
            break;
        case 2:
            HalfLon = 0.0005;
            break;
        case 3:
            HalfLon = 0.00075;
            break;
    }

    long LoLat = ((long)Hi >> 4) & 1023;
    long HiLat = ((long)Hi >> 24) & 255;
    if (HiLat > 180)
    {
        Lat = -180;
        Lon = -180;
        return false;
    }
    if (HiLat > 90)
    {
        HiLat = HiLat - 180;
        Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
    }
    else
    {
        Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
    }

    long LoLon = ((long)Hi >> 14) & 1023;
    long HiLon = ((long)Hi >> 32);
    if (HiLon > 360)
    {
        Lat = -180;
        Lon = -180;
        return false;
    }
    if (HiLon > 180)
    {
        HiLon = HiLon - 360;
        Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
    }

```

```

    }
    else
    {
        Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
    }

    return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}

```

Podsumowanie

Wszystkie założenia zostały spełnione. Stosując zaprezentowaną powyżej logikę możemy zwiększyć dokładność kodu pocztowego dzieląc powierzchnie IPCS na mniejsze części i dokładając kolejne znaki w celu zwiększenia ilości wariacji.